PROYECTO INTEGRADOR FINAL: EXPOTÉCNICA 2026

PARTE 2: Análisis de Riesgos y Modelado de Amenazas

ALUMNOS: ACOSTA Leonel- ESCALADA Facundo- MARECO Martiniano

PROFESORA: ALVARENGA Noelia

INSTITUCION: E.P.E.T N°1 UNESCO

CURSO: 5E

AÑO: 2026

PROYECTO INTEGRADOR FINAL: EXPOTÉCNICA 2026

PARTE 2: Análisis de Riesgos y Modelado de Amenazas

Objetivo

Analizar la arquitectura de la aplicación elegida e identificar al menos **3 vulnerabilidades hipotéticas** o vectores de ataque potenciales que podrían afectar su sistema, documentando cómo mitigarlas (aplicar *Hardening*).

IMPORTANTE (Metodología de Evaluación y Defensa): Todo el progreso, análisis y documentación de esta etapa debe volcarse en el **Tablero de Trello** de su equipo. La defensa oral y técnica conmigo se realizará **directamente desde Trello**, revisando las tarjetas, estados y archivos/enlaces adjuntos, sumado a la demostración en vivo de su aplicación.

Requisitos de Entrega (Repositorio + Trello)

1. **Creación en GitHub:**
   - Crear en la raíz del repositorio el archivo **SECURITY.md** con el Modelado de Amenazas redactado en formato Markdown.
2. **Integración en Trello (Obligatorio para la Defensa):**
   - **Tarjeta de Trello:** Crear una tarjeta específica dentro de la lista correspondiente (ej. *En Proceso* / *Revisión de Seguridad*) titulada: **[Seguridad] Análisis de Riesgos y SECURITY.md**.
   - **Documento Adjunto / Enlace:** Adjuntar en esa tarjeta de Trello el enlace directo al archivo SECURITY.md de GitHub o el documento descargable con el análisis de riesgos.
   - **Subtareas / Checklists:** Incluir en la tarjeta una lista de verificación con los 3 riesgos identificados para ir marcando el estado de análisis y mitigación a medida que avanzan.
   - **Demostración Práctica:** Durante la revisión/defensa, deberán mostrar el sistema funcionando y cómo la mitigación implementada previene la vulnerabilidad planteada.

Estructura del Análisis (SECURITY.md)

Para cada uno de los 3 riesgos, deben detallar:

- **Riesgo / Amenaza:** Qué vulnerabilidad o vector de ataque identifican en su app.
- **Impacto:** Consecuencia real sobre el sistema, los datos o los usuarios si el ataque tiene éxito.
- **Mitigación (Hardening):** Solución técnica aplicada o a aplicar en el código (Python/Flask o MongoDB).

Ejemplos Guía según la Temática de su Grupo Opción A: Para los grupos de "Registro de Estudiantes" (Grupos 5 y 6)

- **Riesgo:** Inyección de consultas NoSQL u Operadores de MongoDB (como $ne o $gt) en el formulario de búsqueda de alumnos para intentar saltearse validaciones o extraer legajos no autorizados.
- **Impacto:** Filtración masiva de datos personales de los estudiantes.
- **Mitigación (Hardening):** Castear y filtrar todos los parámetros de búsqueda a tipo string o aplicar expresiones regulares estrictas en Python antes de realizar la consulta en MongoDB.

Opción B: Para los grupos de "Quiz / Trivia" y "Juegos" (Grupos 3, 4, 7 y 8)

- **Riesgo:** Modificación indebida del ranking o puntuaciones enviando peticiones manipuladas HTTP POST/JSON directamente desde la consola del navegador (Developer Tools) o con herramientas como Postman.
- **Impacto:** Alteración de la integridad del ranking de mejores puntuaciones.
- **Mitigación (Hardening):** Recalcular y validar la puntuación estrictamente en el **Backend (Python/Flask)** y **nunca** confiar en los valores numéricos enviados desde el Frontend (JavaScript).

Opción C: Para el Grupo 2 ("Catálogo de Canciones / Libros")

- **Riesgo:** Inyección de código malicioso (Cross-Site Scripting - XSS) a través del campo de comentarios o entradas de catálogo, ingresando etiquetas HTML o scripts (ej. <script>alert('Hack')</script>).
- **Impacto:** Ejecución de scripts no deseados en el navegador de otros usuarios que naveguen el catálogo.
- **Mitigación (Hardening):** Sanitizar y limpiar todos los datos ingresados por el usuario utilizando funciones/librerías de sanitización en Python antes de guardarlos en MongoDB o renderizarlos en Jinja2.

Opción D: Para el Grupo 1 ("Lista de Tareas")

- **Riesgo:** Acceso/Modificación de tareas pertenecientes a otros usuarios mediante la manipulación de IDs en las rutas HTTP de la aplicación (vulnerabilidad tipo IDOR / Broken Access Control).
- **Impacto:** Manipulación, eliminación o visualización de información privada de otros usuarios.

- **Mitigación (Hardening):** Verificar en la sesión activa del Backend (Flask) que el usuario autenticado sea el verdadero propietario del registro antes de ejecutar la consulta en la base de datos.

Ejemplo de Plantilla Base para el archivo SECURITY.md

Modelado de Amenazas - Grupo [Número de Grupo]

Proyecto: [Nombre del Proyecto]

1. Amenaza 1: [Título corto del riesgo]

- **Descripción del Riesgo:** ...

- **Impacto:** ...

- **Mitigación / Hardening:** ...

2. Amenaza 2: [Título corto del riesgo]

- **Descripción del Riesgo:** ...

- **Impacto:** ...

- **Mitigación / Hardening:** ...

3. Amenaza 3: [Título corto del riesgo]

- **Descripción del Riesgo:** ...

- **Impacto:** ...

- **Mitigación / Hardening:** ...

DETALLE EJEMPLO:

1. Amenaza 1: [Título corto del riesgo]

- **Descripción del Riesgo:** (¿Cuál es el problema? Ej: *Inyección NoSQL en el buscador*, *Modificación de puntajes desde Postman* o *Falta de validación en los inputs*).

- **Impacto:** (¿Qué pasa si el atacante explota esto? Ej: *Se filtran datos privados de los alumnos* o *Se arruina la integridad del ranking*).

- **Mitigación / Hardening:** (¿Cómo lo solucionan en el código de Python/Flask o MongoDB? Ej: *Validar y castear los campos a string antes de consultar la base de datos*[cite: 3]).

DESARROLLO:

Amenaza 1 — Inyección NoSQL o búsqueda manipulada

Riesgo / amenaza: una persona podría intentar introducir operadores de MongoDB o patrones especiales en el parámetro de búsqueda para modificar la consulta y obtener resultados no esperados.

Punto relacionado en el código: `GET /estudiantes?q=...` y `GET /api/estudiantes?q=...` reciben texto de búsqueda. En `RepositorioEstudiantes.cargar()` el texto se inserta en una expresión `$regex` de MongoDB.

Impacto: si la entrada llegara a interpretarse como operador o patrón no previsto, podría ampliarse la búsqueda y exponerse información personal de estudiantes. Una consulta de búsqueda descontrolada también puede consumir recursos.

Mitigación / hardening:

- **Control ya presente:** Flask obtiene `q` como valor de texto (`request.args.get`) y `repositorio.py` utiliza `re.escape(texto)` antes de crear `$regex` ; esto hace que metacaracteres recibidos se interpreten literalmente y no como operadores de consulta.

- Mantener una lista blanca de campos consultables y construir las consultas en el servidor; nunca aceptar del cliente un documento MongoDB ni operadores como `$where` , `$ne` o `$gt` .

- Añadir un límite de longitud a `q` (por ejemplo, 100 caracteres) y responder `400` si se supera.

- Probar entradas como `{"$ne": null}` , `$gt` , comillas y cadenas largas, verificando que se traten como texto y no amplíen la consulta.

Estado: parcialmente mitigado por el escape existente; el límite de longitud y las pruebas de seguridad quedan pendientes.

Amenaza 2 — Acceso no autenticado y control de acceso insuficiente (IDOR)

Riesgo / amenaza: las rutas de consulta, alta, edición y eliminación de la interfaz y de `/api/estudiantes` no exigen iniciar sesión ni comprueban permisos. Un usuario que conozca o consiga un ID podría solicitar el detalle, modificar o borrar una ficha; además puede consultar el listado completo por la ruta pública.

Impacto: exposición de nombres y correos de alumnos, alteración de datos, altas falsas o eliminación de registros. La dificultad de adivinar un ObjectId no sustituye un control de acceso.

Mitigación / hardening:

- Implementar autenticación para personal autorizado (por ejemplo, Flask-Login con contraseñas almacenadas como hashes fuertes) y proteger todas las rutas web y API con sesión/autorización.
- Definir roles/permisos del contexto escolar. Como el modelo actual no tiene propietario por registro, limitar la administración a personal autorizado; no basar el permiso en que el cliente conozca el `id` .
- Responder `401` para usuario no autenticado y `403` cuando no tenga permiso; probar lectura, edición y borrado con sesión ausente y con rol no autorizado.
- Mantener la base MongoDB accesible solo desde el servidor de aplicación y usar credenciales de base con permisos mínimos si la instalación real sale del entorno local.

Estado: pendiente. El código analizado no implementa usuarios, login ni autorización.

Amenaza 3 — Falsificación de solicitudes entre sitios (CSRF)

Riesgo / amenaza: los formularios web de alta, edición y eliminación usan POST, pero no se observa token CSRF. La confirmación JavaScript de eliminación sirve como aviso al usuario, no como protección de seguridad. Cuando se incorporen sesiones autenticadas, un sitio externo podría inducir al navegador de una persona con sesión abierta a enviar cambios no deseados.

Impacto: creación, modificación o eliminación de registros sin que el personal quiera realizar la acción. En el estado actual, las operaciones son públicas por falta de login; la ausencia de CSRF seguirá siendo un riesgo al añadir autenticación mediante cookies.

Mitigación / hardening:

- Incorporar protección CSRF (por ejemplo, Flask-WTF/CSRFProtect) a todos los formularios que cambian datos y validar también las operaciones web sensibles.

- Al usar cookies de sesión, configurar Secure , HttpOnly y SameSite=Lax o Strict , junto con HTTPS en despliegue; estas opciones complementan, pero no reemplazan, el token CSRF.

- Mantener las acciones que modifican datos en POST/PUT/PATCH/DELETE y no realizarlas mediante GET.

- Probar que una solicitud sin token válido falle y que un formulario legítimo con token funcione. No considerar confirm() del navegador como mitigación suficiente.

Estado: pendiente. No se encontró token CSRF en los formularios analizados.